루트해시 서비스

소개

루트해시 서비스(Root Hash Service)는 Oasis Core 네트워크에서 런타임 상태 커밋을 처리하고 검증하는 핵심 서비스입니다. 이 서비스는 런타임 계층에서 실행된 트랜잭션의 결과를 합의 계층에 안전하게 커밋하고, 실행 결과의 일관성을 보장하며, 디스크레판시(불일치) 탐지와 해결을 담당합니다.

주요 특징

- **상태 루트 관리**: 런타임 상태의 Merkle 루트를 안전하게 관리
- **디스크레판시 탐지**: 실행 노드 간 결과 불일치를 자동으로 감지
- **합의 기반 해결**: 불일치 발생 시 다수결 원칙으로 올바른 결과 결정
- **증명 기반 검증**: TEE(Trusted Execution Environment) 기반 실행 증명 검증

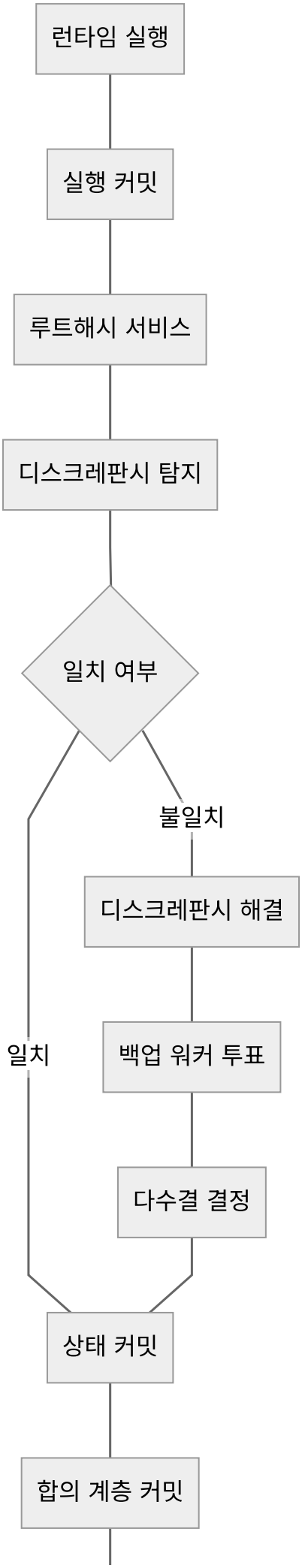
관련 프로세스

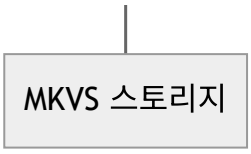
- **합의 프로세스**: 루트해시 트랜잭션의 합의와 커밋 처리
- **스케줄러 서비스**: 실행 위원회 구성과 트랜잭션 스케줄링
- **런타임 호스트**: 런타임 실행 환경과 상태 관리
- **스토리지 서비스**: MKVS 기반 상태 저장과 검색

아키텍처

전체 구조

루트해시 서비스는 런타임 계층과 합의 계층 사이의 브리지 역할을 하며, 런타임 실행 결과의 무결성을 보장합니다.


런타임 실행
실행 커밋
루트해시 서비스
디스크레판시 탐지
일치 여부
불일치
디스크레판시 해결
일치
백업 워커 투표
다수결 결정
상태 커밋
합의 계층 커밋



주요 컴포넌트

커밋 풀 (Commitment Pool)

역할: 실행 노드들의 커밋을 수집하고 검증

주요 기능:

- 실행 커밋 수집과 검증
- 스케줄러 순위 기반 우선순위 관리
- 디스크레판시 탐지 로직 실행
- 투표 집계와 다수결 결정

상태 루트 관리자

역할: 런타임 상태 루트의 생성과 검증

주요 기능:

- Merkle 상태 루트 계산
- I/O 루트와 상태 루트 분리 관리
- 이전 상태와의 연결성 검증
- 상태 변경 추적

디스크레판시 탐지기

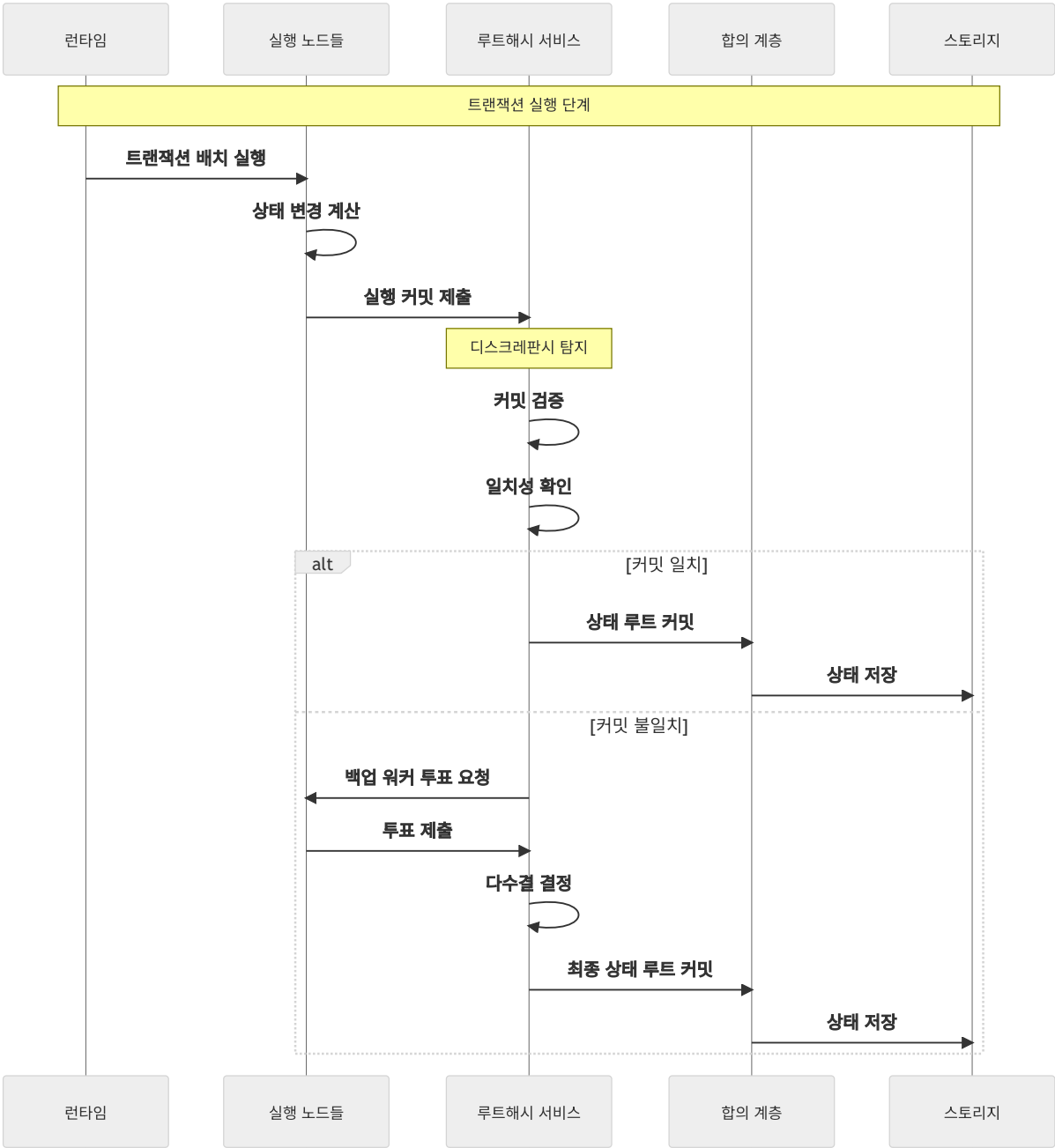
역할: 실행 결과 불일치 감지와 해결 프로세스 관리

주요 기능:

- 실행 커밋 간 차이점 분석
- 디스크레판시 발생 시 해결 모드 전환

- 백업 워커 투표 수집
- 다수결 기반 최종 결과 결정

상태 커밋 과정


런타임
실행 노드들
루트해시 서비스
합의 계층
스토리지
트랜잭션 실행 단계
트랜잭션 배치 실행
상태 변경 계산
실행 커밋 제출
디스크레판시 탐지
커밋 검증
일치성 확인
alt
[커밋 일치]
상태 루트 커밋
상태 저장
[커밋 불일치]
백업 워커 투표 요청
투표 제출
다수결 결정
최종 상태 루트 커밋
상태 저장
런타임
실행 노드들
루트해시 서비스
합의 계층
스토리지

상태 루트 검증과 합의 메커니즘

상태 루트 구조

Oasis Core는 런타임 상태를 두 개의 분리된 Merkle 트리로 관리합니다:

상태 루트 (State Root)

- 런타임의 내부 상태를 나타내는 Merkle 루트
- 스마트 컨트랙트 상태, 계정 잔액 등 포함
- 트랜잭션 실행 후 변경된 상태 반영

I/O 루트 (I/O Root)

- 입출력 데이터의 Merkle 루트
- 트랜잭션 입력과 실행 결과 포함
- 실행 과정의 투명성 보장

```
// 블록 헤더의 상태 루트 구조
type Header struct {
Version  uint16  `json:"version"`
Namespace common.Namespace `json:"namespace"`
Round  uint64  `json:"round"`
Timestamp Timestamp `json:"timestamp"`
HeaderType HeaderType `json:"header_type"`
PreviousHash hash.Hash `json:"previous_hash"`
IORoot hash.Hash `json:"io_root"`  // I/O Merkle 루트
StateRoot hash.Hash `json:"state_root"`  // 상태 Merkle 루트
```

```
MessagesHash hash.Hash `json:"messages_hash"`  // 메시지 해시
InMessagesHash hash.Hash `json:"in_msgs_hash"`  // 입력 메시지 해시
}
```

합의 메커니즘

1단계: 실행 커밋 수집

실행 노드들이 트랜잭션 배치를 실행한 후 커밋을 제출합니다:

```
// 실행 커밋 구조
type ExecutorCommitment struct {
NodeID signature.PublicKey `json:"node_id"`
Header ComputeResultsHeader `json:"header"`
Messages []message.Message `json:"messages,omitempty"`
}

// 커밋 헤더
type ComputeResultsHeader struct {
Round  uint64  `json:"round"`
PreviousHash hash.Hash `json:"previous_hash"`
IORoot *hash.Hash `json:"io_root,omitempty"`
StateRoot *hash.Hash `json:"state_root,omitempty"`
MessagesHash *hash.Hash `json:"messages_hash,omitempty"`
Failure FailureIndicatorKind `json:"failure,omitempty"`
```

```
}
```

2단계: 커밋 검증

루트해시 서비스는 수신된 커밋의 유효성을 검증합니다:

- **서명 검증**: 커밋 제출자의 디지털 서명 확인
- **TEE 증명 검증**: SGX 환경에서 실행된 경우 원격 증명 확인
- **이전 블록 연결성**: 이전 블록과의 올바른 연결 확인
- **메시지 일관성**: 포함된 메시지의 해시 일치성 확인

3단계: 디스크레판시 탐지

커밋들 간의 일치성을 확인하여 디스크레판시를 탐지합니다:

```
// 디스크레판시 탐지 로직
func (p *Pool) processCommitments(c *scheduler.Committee, allowedStragglers  uin
// 투표 집계
votes := make(map[hash.Hash]int)
for  _, n := range  c.Members {
if  vote, ok := sc.Votes[n.PublicKey];  ok && vote != nil {
votes[*vote]++
}
}
// 디스크레판시 탐지
if  len(votes) > 1 || failures > int(allowedStragglers) {
return  nil, ErrDiscrepancyDetected
```

```
}
return  sc, nil
}
```

디스크레판시 탐지와 해결 프로세스

디스크레판시 탐지

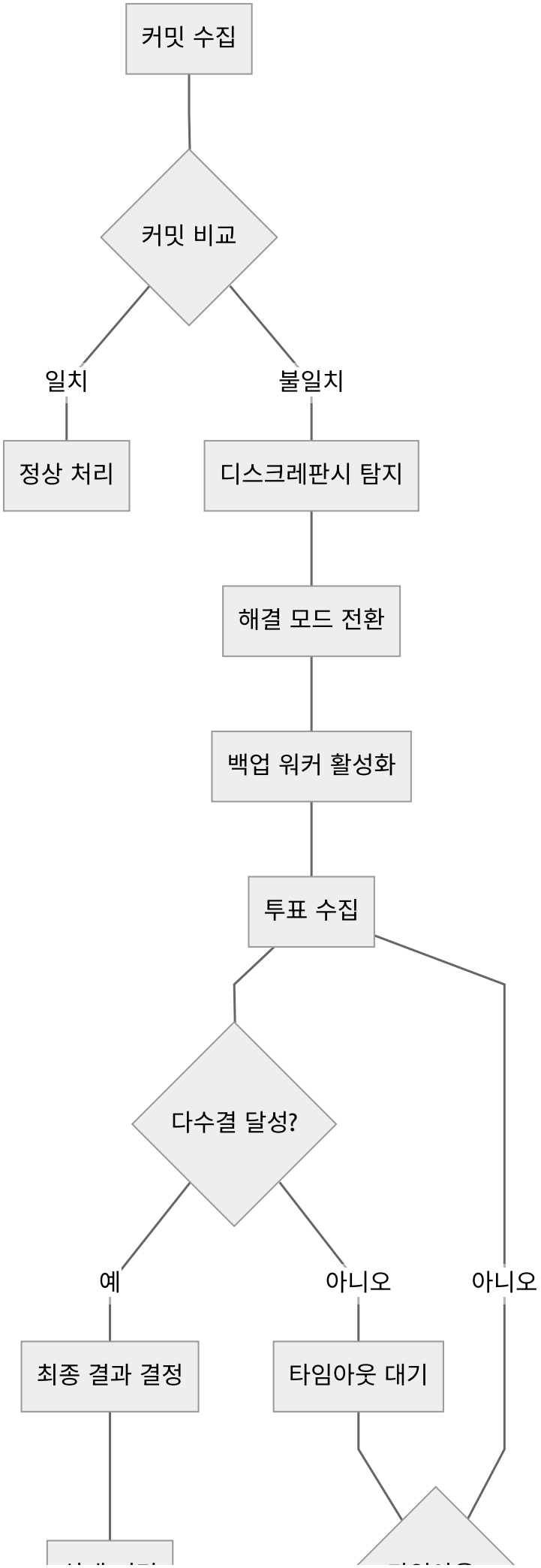
디스크레판시는 다음 상황에서 발생합니다:

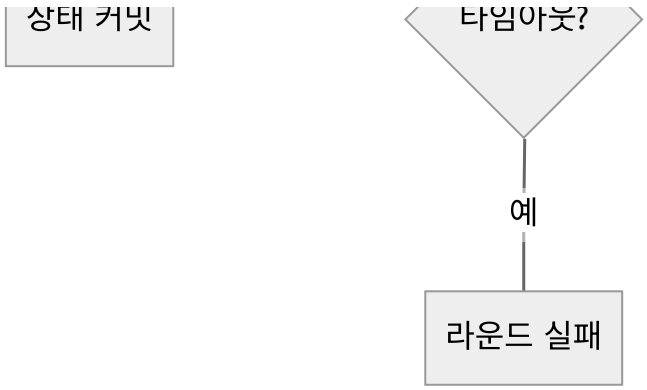
실행 결과 불일치

- 서로 다른 상태 루트 제출
- 서로 다른 I/O 루트 제출
- 서로 다른 메시지 해시 제출

실행 실패 보고

- 일부 노드가 실행 실패 보고
- 허용된 실패 노드 수 초과
- 타임아웃 발생


커밋 수집
커밋 비교
일치
불일치
정상 처리
디스크레판시 탐지
해결 모드 전환
백업 워커 활성화
투표 수집
다수결 달성?
예
아니오
아니오
최종 결과 결정
타임아웃 대기



디스크레판시 해결 과정

1단계: 해결 모드 전환

디스크레판시가 탐지되면 해결 모드로 전환됩니다:

```
// 디스크레판시 해결 모드 전환
switch err {
case ErrDiscrepancyDetected:
// 디스크레판시 해결 모드로 전환
p.Discrepancy = true
// 다른 순위의 커밋 제거
for r := range p.SchedulerCommitments {
if r != p.HighestRank {
delete(p.SchedulerCommitments, r)
}
}
}
```

2단계: 백업 워커 투표

백업 워커들이 독립적으로 트랜잭션을 실행하고 투표합니다:

- **독립 실행**: 백업 워커들이 동일한 트랜잭션 배치를 실행
- **투표 제출**: 각자의 실행 결과를 투표로 제출
- **투표 집계**: 루트해시 서비스가 투표를 수집하고 집계

3단계: 다수결 결정

수집된 투표를 바탕으로 최종 결과를 결정합니다:

```
// 다수결 결정 로직
required := total/2 + 1  // 과반수 요구

// 가장 많은 투표를 받은 커밋 찾기
var (
hash hash.Hash
best  int
)
for  h, v := range  votes {
if  v > best {
hash = h
best = v
}
}

// 과반수 달성 확인
if  best >= required {
```

```
// 최종 결과로 채택
return  sc, nil
}
```

디스크레판시 해결 결과

성공적 해결

- 과반수 합의 달성 시 해당 결과 채택
- 올바른 결과를 제출한 노드들의 평판 향상
- 잘못된 결과를 제출한 노드들의 평판 하락

해결 실패

- 과반수 합의 실패 시 라운드 실패 처리
- 빈 블록 생성으로 라운드 진행
- 실패한 라운드는 상태 변경 없이 처리

API 및 인터페이스

루트해시 백엔드 인터페이스

```
// Backend는 루트해시 백엔드의 주요 인터페이스입니다
type Backend interface {
// GetLatestBlock은 특정 런타임의 최신 블록을 반환합니다
GetLatestBlock(ctx context.Context, request *RuntimeRequest) (*block.Block, erro
// GetRuntimeState는 런타임의 현재 상태를 반환합니다
```

```
GetRuntimeState(ctx context.Context, request *RuntimeRequest) (*RuntimeState, er

// GetRoundRoots는 특정 라운드의 상태 루트와 I/O 루트를 반환합니다

GetRoundRoots(ctx context.Context, request *RoundRootsRequest) (*RoundRoots, err

// WatchBlocks는 새로운 블록 스트림을 반환합니다

WatchBlocks(ctx context.Context, runtimeID common.Namespace) (<-chan *AnnotatedB

// WatchEvents는 루트해시 이벤트 스트림을 반환합니다

WatchEvents(ctx context.Context, runtimeID common.Namespace) (<-chan *Event, pub

}
```

커밋 제출 인터페이스

```
// ExecutorCommit은 실행 커밋 제출을 위한 구조체입니다

type ExecutorCommit struct {

ID common.Namespace `json:"id"`  // 런타임 ID

Commits []commitment.ExecutorCommitment `json:"commits"`  // 실행 커밋 목록

}


// 실행 커밋 트랜잭션 생성

func  NewExecutorCommitTx(nonce  uint64, fee *transaction.Fee, runtimeID common.

return  transaction.NewTransaction(nonce, fee, MethodExecutorCommit, &ExecutorCo

ID: runtimeID,

Commits: commits,

})
```

```
}
```

gRPC API

루트해시 서비스 정의

```
service RootHashService {
// GetLatestBlock은 최신 블록을 조회합니다
rpc  GetLatestBlock(RuntimeRequest) returns (Block);
// GetRuntimeState는 런타임 상태를 조회합니다
rpc  GetRuntimeState(RuntimeRequest) returns (RuntimeState);
// GetRoundRoots는 라운드별 루트를 조회합니다
rpc  GetRoundRoots(RoundRootsRequest) returns (RoundRoots);
// WatchBlocks는 블록 스트림을 구독합니다
rpc  WatchBlocks(RuntimeRequest) returns (stream AnnotatedBlock);
}
```

메시지 타입

```
message RuntimeRequest {
bytes  runtime_id = 1; // 런타임 ID
int64  height = 2; // 블록 높이 (0은 최신)
}
```

```
message RoundRootsRequest {
bytes  runtime_id = 1; // 런타임 ID
int64  height = 2; // 블록 높이
uint64  round = 3; // 라운드 번호
}

message RoundRoots {
bytes  state_root = 1; // 상태 루트
bytes  io_root = 2; // I/O 루트
}
```

설정 및 구성

루트해시 설정 구조

```
roothash:
# 디버그 설정
debug_do_not_suspend_runtimes: false  # 런타임 일시정지 비활성화
debug_bypass_stake: false  # 스테이킹 검증 우회
# 메시지 제한
max_runtime_messages: 256  # 최대 런타임 메시지 수
max_in_runtime_messages: 128  # 최대 입력 메시지 수
# 증거 관리
```

```
max_evidence_age: 100  # 증거 최대 보관 라운드
max_past_roots_stored: 1200  # 과거 루트 최대 보관 수
```

주요 설정 옵션

옵션 이름	타입	기본값	설명
`max_runtime_messages`	uint32	256	라운드당 최대 런타임 메시지 수
`max_in_runtime_messages`	uint32	128	큐에 저장할 최대 입력 메시지 수
`max_evidence_age`	uint64	100	증거 제출 가능한 최대 라운드 수
`max_past_roots_stored`	uint64	1200	합의 상태에 저장할 과거 루트 수
`debug_do_not_suspend_runtimes`	bool	false	런타임 일시정지 비활성화

가스 비용 설정

```
roothash:
gas_costs:
compute_commit: 1000  # 실행 커밋 제출 비용
proposer_timeout: 1000  # 제안자 타임아웃 비용
evidence: 1000  # 증거 제출 비용
submit_msg: 1000  # 메시지 제출 비용
```

코드 예제

기본 사용법

```go
package main

import (
"context"
"fmt"
"log"
"github.com/oasisprotocol/oasis-core/go/common"
"github.com/oasisprotocol/oasis-core/go/roothash/api"
)

func main() {
// 루트해시 클라이언트 생성
client, err := api.NewRootHashClient("unix:/tmp/oasis-node-roothash.sock")
if err != nil {
log.Fatal("루트해시 클라이언트 생성 실패:", err)
}
ctx := context.Background()
runtimeID := common.NewTestNamespaceFromSeed([]byte("test-runtime"), 0)
// 최신 블록 조회
block, err := client.GetLatestBlock(ctx, &api.RuntimeRequest{
RuntimeID: runtimeID,
```

```
Height: 0, // 0은 최신 블록
})
if err != nil {
log.Fatal("최신 블록 조회 실패:", err)
}
fmt.Printf("최신 블록 라운드: %d\n", block.Header.Round)
fmt.Printf("상태 루트: %s\n", block.Header.StateRoot)
fmt.Printf("I/O 루트: %s\n", block.Header.IORoot)
}
```

런타임 상태 조회

```
// 런타임 상태 정보 조회
func getRuntimeState(client api.Backend, runtimeID common.Namespace) error {
ctx := context.Background()
// 런타임 상태 조회
state, err := client.GetRuntimeState(ctx, &api.RuntimeRequest{
RuntimeID: runtimeID,
Height: 0,
})
if err != nil {
return fmt.Errorf("런타임 상태 조회 실패: %w", err)
}
```

```go
fmt.Printf("런타임 ID: %s\n", state.Runtime.ID)
fmt.Printf("일시정지 상태: %t\n", state.Suspended)
fmt.Printf("최신 블록 높이: %d\n", state.LastBlockHeight)
fmt.Printf("최신 정상 라운드: %d\n", state.LastNormalRound)
// 위원회 정보 출력
if state.Committee != nil {
fmt.Printf("위원회 크기: %d\n", len(state.Committee.Members))
fmt.Printf("위원회 유효 기간: %d\n", state.Committee.ValidFor)
}
return nil
}
```

블록 모니터링

```go
// 새로운 블록 모니터링
func watchBlocks(client api.Backend, runtimeID common.Namespace) error {
ctx := context.Background()
// 블록 스트림 구독
blockCh, sub, err := client.WatchBlocks(ctx, runtimeID)
if err != nil {
return fmt.Errorf("블록 구독 실패: %w", err)
}
defer sub.Close()
```

```
fmt.Printf("런타임 %s의 새로운 블록을 모니터링합니다...\n", runtimeID)
for {
select {
case  block := <-blockCh:
fmt.Printf("새로운 블록: 라운드=%d, 높이=%d, 상태루트=%s\n",
block.Block.Header.Round,
block.Height,
block.Block.Header.StateRoot)
// 디스크레판시 이벤트 확인
if  block.Block.Header.HeaderType == block.RoundFailed {
fmt.Printf("⚠️ 라운드 %d 실패 (디스크레판시 해결 실패)\n",
block.Block.Header.Round)
}
case <-ctx.Done():
return  ctx.Err()
}
}
}
```

실행 커밋 제출

```
// 실행 커밋 제출 (실행 노드용)
func  submitExecutorCommit(client api.Backend, signer signature.Signer, runtimeI
```

```go
ctx := context.Background()

// 실행 결과 헤더 생성

header := &commitment.ComputeResultsHeader{

Round: 100,

PreviousHash: hash.NewFromBytes([]byte("previous-block-hash")),

IORoot: &hash.Hash{}, // 실제 I/O 루트

StateRoot: &hash.Hash{}, // 실제 상태 루트

MessagesHash: &hash.Hash{}, // 메시지 해시

}

// 실행 커밋 생성

commit := &commitment.ExecutorCommitment{

NodeID: signer.Public(),

Header: header,

Messages: []message.Message{}, // 런타임 메시지

}

// 커밋 서명

if err := commit.Sign(signer, runtimeID); err != nil {

return fmt.Errorf("커밋 서명 실패: %w", err)

}

// 트랜잭션 생성 및 제출

tx := api.NewExecutorCommitTx(1, &transaction.Fee{}, runtimeID, []commitment.Exe

// 합의 계층에 트랜잭션 제출

if err := consensus.SubmitTx(ctx, tx); err != nil {
```

```
return fmt.Errorf("커밋 제출 실패: %w", err)
}
fmt.Println("실행 커밋이 성공적으로 제출되었습니다")
return nil
}
```

CLI 사용 예제

```
# 런타임 상태 확인
oasis-node roothash show-runtime-state --runtime-id <runtime-id>

# 최신 블록 조회
oasis-node roothash show-latest-block --runtime-id <runtime-id>

# 특정 라운드의 루트 조회
oasis-node roothash show-round-roots --runtime-id <runtime-id> --round 100

# 루트해시 이벤트 모니터링
oasis-node roothash watch-events --runtime-id <runtime-id>

# 실행 커밋 제출
oasis-node roothash submit-executor-commit --runtime-id <runtime-id> --commit
```

모니터링 및 메트릭

주요 메트릭

| 메트릭 이름 | 타입 | 설명 |

|----------|------|------|

| `oasis_roothash_blocks_total` | Counter | 총 처리된 블록 수 |

| `oasis_roothash_rounds_total` | Counter | 총 처리된 라운드 수 |

| `oasis_roothash_discrepancies_total` | Counter | 탐지된 디스크레판시 수 |

| `oasis_roothash_round_duration_seconds` | Histogram | 라운드 처리 시간 |

| `oasis_roothash_commitments_total` | Counter | 수신된 커밋 수 |

| `oasis_roothash_failed_rounds_total` | Counter | 실패한 라운드 수 |

이벤트 모니터링

```
# 루트해시 이벤트 스트림 확인
curl -s http://localhost:9001/api/v1/roothash/events | jq

# 응답 예시
{
"height": 12345,
"runtime_id": "8000000000000000000000000000000000000000000000000000000000000000
"finalized": {
"round": 100
}
```

```
}
```

헬스 체크

```
# 루트해시 서비스 상태 확인
curl http://localhost:9001/api/v1/roothash/status

# 응답 예시
{
"latest_height": 12345,
"latest_round": 100,
"runtime_id": "8000000000000000000000000000000000000000000000000000000000000000
"is_syncing": false,
"committee_size": 5,
"discrepancy_detected": false
}
```

문제 해결

일반적인 문제

문제 1: 디스크레판시 빈발

증상: 디스크레판시가 자주 발생하여 라운드 처리가 지연됨

원인:

- 실행 노드 간 상태 불일치
- 네트워크 지연으로 인한 타임아웃
- 악의적 노드의 잘못된 커밋 제출

해결 방법:

1. 실행 노드 상태 동기화 확인

```
oasis-node roothash show-runtime-state --runtime-id <runtime-id>
```

2. 네트워크 연결 상태 점검
3. 노드 로그에서 실행 오류 확인

```
grep "discrepancy" /var/log/oasis/node.log
```

문제 2: 커밋 제출 실패

증상: 실행 노드가 커밋을 제출하지 못함

원인:

- 위원회에 포함되지 않음
- 서명 검증 실패
- TEE 증명 검증 실패

해결 방법:

1. 위원회 멤버십 확인

```
oasis-node scheduler show-committee --runtime-id <runtime-id>
```

2. 노드 등록 상태 확인

```
oasis-node registry node list --node-id <node-id>
```

3. TEE 설정 검증 (SGX 환경인 경우)

문제 3: 라운드 타임아웃

증상: 라운드가 타임아웃되어 빈 블록이 생성됨

원인:

- 실행 노드 응답 지연
- 네트워크 분할
- 실행 노드 장애

해결 방법:

1. 실행 노드 상태 확인
2. 네트워크 연결성 점검
3. 타임아웃 설정 조정 (필요시)

로그 분석

```
# 루트해시 관련 로그 확인
journalctl -u oasis-node -f | grep roothash

# 디스크레판시 관련 로그 확인
grep "discrepancy\|commitment" /var/log/oasis/node.log
```

```
# 커밋 풀 상태 로그 확인
grep "pool\|commitment" /var/log/oasis/node.log | tail -20

# 라운드 실패 로그 확인
grep "round.*failed\|timeout" /var/log/oasis/node.log
```

성능 튜닝

커밋 풀 최적화

```
roothash:
# 더 많은 실패 노드 허용 (신중하게 설정)
allowed_stragglers: 1
# 타임아웃 조정
round_timeout: "30s"
```

메시지 처리 최적화

```
roothash:
# 메시지 큐 크기 조정
max_in_runtime_messages: 256
# 배치 크기 최적화
max_runtime_messages: 512
```

참조

관련 문서

- 합의 프로세스
- 스케줄러 서비스
- 런타임 호스트
- MKVS 스토리지
- 아키텍처 개요

소스 코드

- 루트해시 API
- 커밋 풀 구현
- 블록 구조

외부 리소스

- Oasis Core 공식 문서
- 런타임 개발 가이드
- Merkle Tree 개념

마지막 업데이트: 2024년 1월

작성자: Oasis Core 문서화 팀

검토자: 루트해시 개발팀